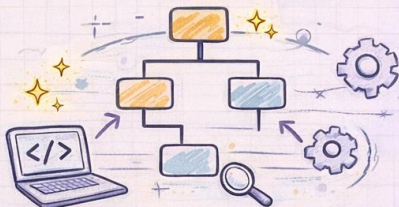


Angular Tutorial

Zero to MASTER



Component Tree

Handwritten Notes to
Master Angular



Pankaj Ikhar

Solution Architect & Mentor

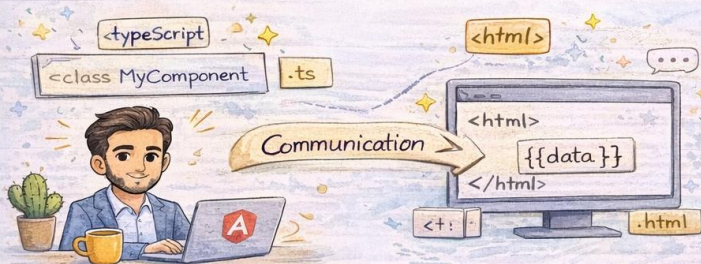
Connect with me!

For interview preparation, short & long term
mentorship, and design & architectural consulting.

What is Data Binding?

⇒ Communication between Typescript code of your component and the template (html) that the user see

Data Binding = Communication



We get different ways of communication.

Syntax for String Interpolation

`{{ data }}`

You put your data that will come from TS file in double curly braces.



Examples of Data Binding in Angular

➤ Eg of String Interpolation

.ts

```
export class MyComponent {  
  name: string = 'deepa';  
}
```

<p> My name is {{ name }} </p>

Output: My name is deepa



➤ Eg of Property Binding

.ts

```
export class MyComponent {  
  isAdmin: boolean = true;  
}
```

Use property to bind/
button based on logic in TS file

Button is enabled



.html

<button [disabled]="isAdmin">Add Button

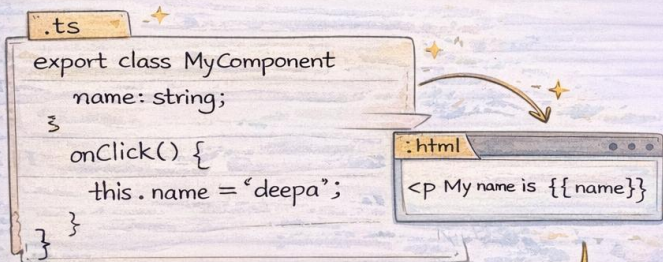
- If isAdmin = true → Button is enabled
- If isAdmin = false → Button is disabled

Event Binding in Angular

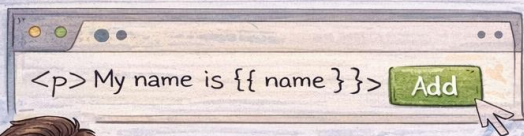
→ Allows your component to react to users' actions such as button clicks, key strokes, etc.

Like property binding, use `[]` square brackets

> On click of button, show something:



> In html



★ Before clicking Add: name will be blank

★ After clicking Add: name will be set to:

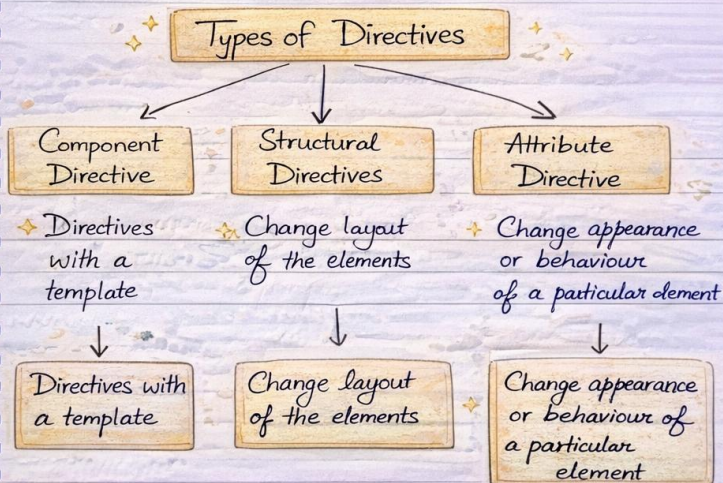
== `deepa`

What are Directives?

→ Directives are instructions in the DOM!

Basically, These are custom HTML attributes which tell Angular to change the style or behaviour of Dom elements.

Components are building blocks of Angular applications. Similarly, directives are also building blocks of Angular Applications.



1. Component Directives

⇒ Directives must be declared in Angular Modules in the same manner as Components

- * Directive in Angular is a re-usable component
- * The other directives such as Structural and Attribute do not have templates

It is decorated with @Component decorator

Example: "change-text" directive

```
import { Directive, ElementRef } from '@angular/core';
```

```
@Directive({  
  selector: ["changeText"]  
})
```

```
export class ChangeTextDirective {}
```

```
@Directive({  
  selector: ["changeText"]  
})  
export class ChangeTextDirective {  
  constructor ( element : ElementRef ) {  
    Element.nativeElement.innerText =  
      "Text is changed."  
  }  
}
```


app.component.html

```
<span changeText> Welcomo. </span>
```



- * In above eg, there is class called `ChangeText Directive` and a constructor which takes the element of type `ElementRef`, which is mandatory.
- * The element has all the details to which `Change Directive` is applied.

2. Structural Directives :



- * Structural Directives are responsible for changing structure of DOM
- * They work by adding or removing element from DOM.
- * The Structural directive always starts with *

✦ Three most popular structural Directives ✦

- 1) * `ngIf`
- 2) * `ngFor`
- 3) * `ngSwitch`

All of them work same as *If, *For or *Switch respectively.

Except that you use it for DOM Tree in html template

Eg>

```
<div *ngIf="*info" class="name">
```

```
<div *ngFor="let movie of movies"></div>
```

```
<ng-template #loggedOut>
```

```
<{movie.name}> </div>
```

3. 9. Attribute Directives :



- * Attribute Directives manipulate DOM by changing its behaviour and appearance.
- * It is used to show or hide elements or dynamically change behaviour of component
- * Built In Attributes are ngClass etc.

Pankaj Ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

What are Modules In Angular?

Angular doesn't scan automatically all your components, directives etc.

You have to tell Angular what Component, directive, & Service your project have or you are using

Modules does that Job for you

Module tell Angular list of all components, directives or Services we are using.

It is basically a place where you group together all your component, directive, & Service.

```
@NgModule({  
  declaration: [ AppComponent,  
                XComponent,  
  imports: [ AppRoutingModule, ],  
            BrowserModule ],  
  providers: [ { $service goes here } ],  
  bootstrap: [ AppComponent ]  
  entryComponents: [ ]  
})
```

```
Export class AppModule {
```

1. @NgModule - Angular analyzes ngModules to understand application and its features

You can't use a feature without including it in @NgModule.

2. Imports - Imports are used for importing other modules like, FormsModule, RoutingModule etc and their features into our main Module

3. Providers - It defines all services we are providing in our angular app

All services declare in your app must be inside the Providers of APPMODULE

OR

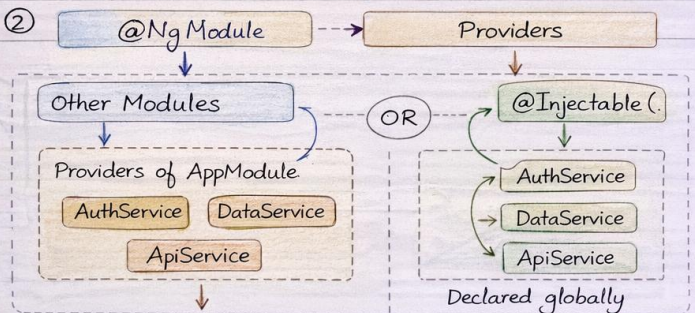
You have one other method to import your services throughout root of application

```
@Injectable ({  
  providedIn: 'root';  
})
```

1. `@NgModule` – Angular analyzes `ngModules` to understand application and its features

You can't use a feature without including it in `@NgModule`.

2. Imports – Imports are used for importing other modules like, `FormsModule`, `RoutingModule` etc and their features into our main Module.
3. Providers – Imports are used for importing other modules like, `FormsModule`, `RoutingModule` etc and their features into our main Module.



3. `Bootstrap[]` – Bootstrap is important for starting your app. It defines which component is available straight in your app at first (i.e. `app-root`).

Pankaj Ikhar | Solution Architect & Mentor

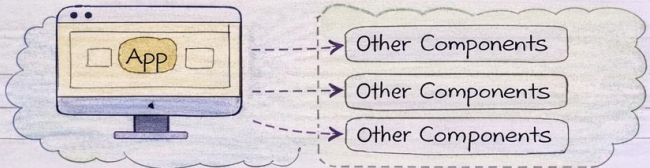
Mentorship • Interview Prep • Architecture Consulting

* If you use @Injectable, you don't have to declare it in providers of Module.

* Remember, `@Injectable({
 providedIn: 'root';
})`

Will come above every
service in Service Component

`Bootstrap[]` — Bootstrap is important for starting your app. It defines which component is available straight in your app at first (i.e. app-root)

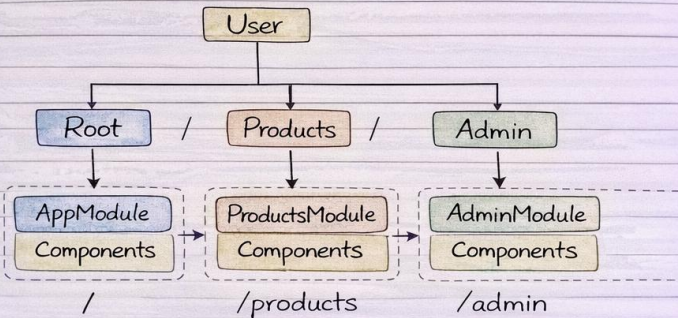


You can include other components also that's why it's array. But it is not likely to add other components generally.

Lazy Loading In Angular

Consider, Our application has 3 routes.

Every routes has a module associated with it that contains their respective component, services and directives.



Here you can see we have 3 routes

- Root
- Products
- Admin

On visiting the root route, we must be loading the respective modules, and then we load other modules when required.

Pankaj Ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

But does it really Happen ??

The answer is **NO**.

Angular actually load all the modules whenever we visit any page



It's like you ordered starter, maincourse and dessert

NO, I will make all starter, maincourse and dessert first then start serving.

And chef is saying,



Starter



Maincourse

You definitely want atleast your starter first, and ask chef to make rest when it's required.

So Lazy Loading does the same for you



In Lazy Loading, the modules are loaded only if the user hit the routes of that particular Module.

Pankaj Ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

Advantages of Lazy Loading In Angular

Page

Page

- * Initially we load a smaller bundle, and module for our root route.

- * We load more modules / bundle as we hit route for following

Load smaller bundles for root route.

- * So our first page loads faster, and it doesn't take much time because bundles are comparatively less to download now.



- * Basically your application size shrinks and it becomes faster.



Faster!



Faster!

Faster First Page Loading

- Good Optimization Can be achieved with Lazy Loading

- * You can read about Preloading Strategy, dynamic importing of Components and render using Component Factory for better Angular Build Optimization



Pankaj Ikhar | Solution Architect & Mentor

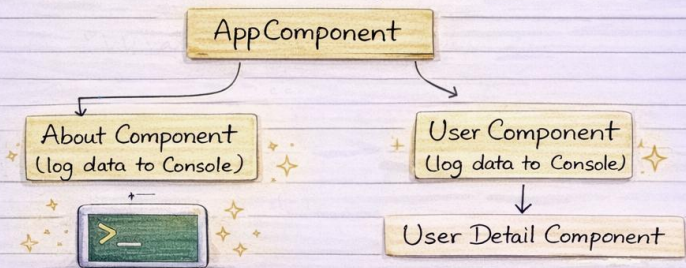
Mentorship • Interview Prep • Architecture Consulting

Services and Dependency Injection.

- * Angular services are singleton objects that get instantiated only once in a lifetime of an application.



- * Consider an application having:



- * We might have some methods in those components

- * Let's say we want to log some data in About Component and UserDetail Component

- * Both logs are same, so basically we are duplicating code in 2 different components

- * Suppose in UserComponent, we also want to access some data of some array of Users and we don't know if we will later use that in other part of application.



Angular Zero to Master Series

Page

Date

Both User Cases

- * Duplication of Code
- * Data Storage

In both cases, we typically use Service

A Service is just another piece in your Angular App, Another class which acts as a central repository as a

central business unit where you can centralize your code.

- * We can create a log Service to centralize our log statements (which is used in 2 components)

- * Service are also useful for Communication between Components

* How to create a Service

- * Simply run command
ng g s service-name
your service will be generated



Pankaj Ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

How do you Access a Service In Your Component ?

- * Let's say you have made service named **Logging Service**
- * You can Simply instantiate your **Logging Service** in component

```
* const service = new new LoggingService();
```

- * Import the **Logging Service**

```
* + service.logStatus(status);
```



... Your method you wanted to call from service in Component

But this is not suggested, Angular provides us more better way to inject Service in Our Components

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

Angular Dependency Injector

Angular provides us more better way to inject service in component. It is Angular dependency
Injector.

- * So, we have dependency on `LoggingService` from our component, and `Dependency Injector` simply injects this dependency instance of any class automatically into our component.
- * We need to only Inform Angular, that we require such an instance.
- * How do we inform that?
- * We add constructor to the component (you'll see constructor present if you have made service through command).
- * Inside constructor, we can bind it to property.

```
constructor(private LoggingService: LoggingService) { ... }
```

(make sure to import `LoggingService` at top!)

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting



Angular Zero to Master Series

Page

Date

Now Angular know we want **LoggingService**

✦ But it doesn't know how to give instance.

✦ We need to provide a service

Add one providers property in @Component decorator :

```
@Component ({  
  selector: ...  
  templateUrl: ...  
  styleUrls: ...  
  providers: [LoggingService] → [Here we have  
  have provided  
})
```

✦ Hierarchical Injector ✦

Angular dependency injector is actually a hierarchical Injector.

✦ It means that if we provide a service in one component, then Angular framework knows how to create instance of service for this component and important, all its child components.



✦ All of them will receive same instance of service ✦

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

✧ Hierarchical Injector ✧

- ✧ App Module → Same Instance of Service is available Application-wide
- ✧ App Component → Same Instance of Service is available for all components (but not for others service)
- ✧ Any Other Component → Same Instance is available for component and all its child components

✧ It works Top to down not down to Top

✧ Suppose if you want to use same Instance in AppComponent to child component

Then you don't use providers in child component

Everything else stays same.

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

Route Guards

What is Route Guards?

✦ They are interface provided by Angular which when implemented allow us to control the accessibility of routes based on condition provided in class implementation of That Interface

✦ Five Types of Route Guards -

1. Can Activate → Can Activate

↓ implement

Auth Guard Service

define class AuthGuard -
Implement CanActivate Method

will call
everytime we provide
to this field



Path: '/user/:id', → Component : HomeComponent
CanActivate : [AuthGuard]

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

Auth Guard Service ✨

```
canActivate (route: ActivatedRouteSnapshot,  
  state: RouterStateSnapshot): Observable<boolean>  
  | Promise<boolean> {  
3  return true; // If navigation happen  
}
```

✨ Now Use this Service.

✨ CanActivateChild - Similar to CanActivate but also prevent access to child routes of given route.

✨ CanLoad - In Angular we implement lazy loading to download only required modules using loadChildren.

✨ If we want to prevent unauthorized users, we can use CanActivate Guard that will do job but also download Module

Now to control navigation as well as prevent downloading of that module we can use CanLoad Guard.

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

✧ ✧ CanLoad | CanDeactivate Guard ✧ ✧

```
✧ canLoad (Route: Route, segments: UrlSegment[]):  
  Observable<boolean | Promise<boolean> > {  
    return true;  
  }  
3. // whether want to download module.
```

✧ Can Deactivate - This guard is useful when you want to prevent user from accidentally navigating away without saving of some undone task.

```
✧ canDeactivate (component: T,  
  currentRoute: ActivatedRouteSnapshot,  
  currentState: RouterStateSnapshot,  
  nextState?:  
  } Observable<boolean | UrlTree | Promise<boolean |  
    UrlTree>>
```

Here is interface that we will be using in component as well as guard

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

// CanDeactivate Service ✨

```
✨ export interface CanComponentDeactivate {
  canDeactivate () : ) => Observable<boolean>
    | Promise<boolean> | boolean ;
}
```

✨ export class CanDeactivate implements
CanComponentDeactivate {

```
✨ CanDeactivate (Component: T,
  currentRoute: ActivatedRouteSnapshot,
  currentState: RouterStateSnapshot,
  nextState?:
  { Observable<boolean> | UrlTree | Promise<boolean> |
    UrlTree>>
  return component canDeactivate ();
}
```

✨ Now I'll implement CanComponentDeactivate
interface in component

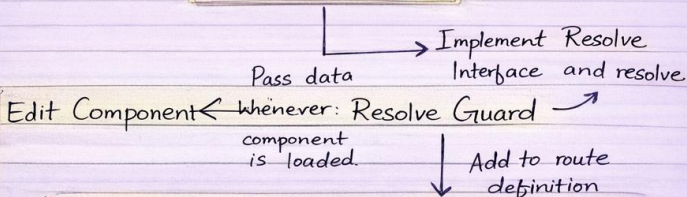
```
✨ export class EditComponent implements OnInit,
  CanComponentDeactivate {
  canDeactivate () : Observable<boolean> | Promise<boolean>
}
```

✦ 5. Resolve Guard ✦

While data communication b/w components, Sometimes data is so heavy, that is not possible to pass data through query params.

✦ To handle this we have Resolve Guard

Resolve Interface



```
{ path: 'id', component: EditComponent,
  resolve: { data: Resolve Guard } }
```

✦ Resolve Guard Service

```
resolve (route, state) {
  return this.fetchService.get(+ route.param['id'])
}
```

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

Difference between Constructor and

ngOnInit in Angular

ngOnInit	Constructor
ngOnInit is just a method on a class, it's completely a choice to use it or not.	Constructor will always be called, whether you implement it or not
ngOnInit is a place to put code we need to execute first as soon as class is Instantiated	It is used for declaration and initialization of classes and sub-classes
Angular calls ngOnInit once it finished creating Component DOM, injected all required dependencies	- It is used to set Dependency Injection - Constructor - first ngOnChanges() - ngOnInit

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

✧ Angular Bootstrap Process ✧

✧ Angular bootstrap process consist of 2 major stages:

- 1) Constructing Component Tree
- 2) Running change detection

✧ Constructor of Component called at 1st stage, when angular constructs component tree

✧ All life-cycle hooks including `ngOnInit` are called in 2nd stage, in change detection phase

✧ Usually a component Initialization logic requires either some DI providers are available input bindings or rendered DOM.

✧ When angular constructs a component tree the root modules injector is already configured so you can inject global dependencies

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

✧ Dependency Injection Mechanics ✧ in Angular

- ✧ When Angular instantiates a child component class, the injector for parent component is already setup, so you can inject providers defined on the parent component including the parent component itself.
- ✧ A constructor is the only method that is called in context of injector, so if you need any dependency that's the only place to get those dependencies.
- ✧ The @Input communication mechanism is processed as part of following change detection phase, so Input bindings not available in constructor.

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

✦ All about @NgModule In Angular ✦

✦ An NgModule class is marked by @NgModule decorator.

@NgModule takes a metadata object

```
{ imports: [],  
  | declarations: [],  
  | providers: [],  
  } exports: [] }
```

which tells components directive and pipes belonging to the module, make some public by using export, so that other components can use them.

✦ @NgModule can also add service providers to application dependency injectors.

Different type of Angular Decorators

- 1) Class Decorators
- 2) Property Decorators
- 3) Method Decorators
- 4) Parameter Decorators

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

✧ Class Decorators ✧

They are top-level decorators that are used to define purpose of classes.

✧ They tell if class is component or Module.

@Component ({ }

export class MyComponent {

@NgModule ({ }

export class MyModule {

✧ @NgModule ({ }

export class MyModule {

✧ Property Decorators

They are used to decorate specific properties inside classes.

For eg - @Input() exampleProp ;

✧ Here with decorator Angular will automatically perform Input binding.

✧ Method Decorators

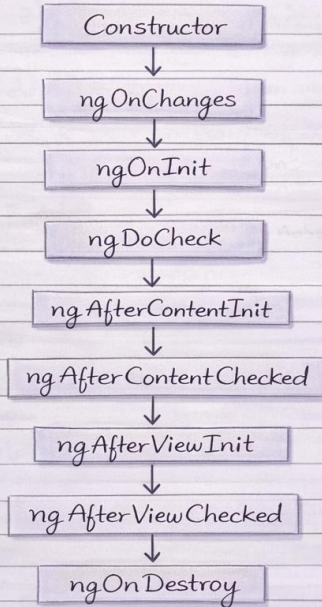
Used to decorate method defined inside own class with functionality.

For eg - @HostListener

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

✧ Life Cycle of Angular Components ✧



Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

Instantiation ✨

Page

Date

★ Method : Constructors()

- It is called on Component/directive Instantiation.
- It allows centralizing the dependency Injections.
- Avoid Subscription in Constructor

★ OnChanges Method

→ `public ngOnChanges(changes: SimpleChanges): void`

It allows to set or reset data-bound input properties (can be called multiple times).

The parameter is HashTable with properties name as key and a SimpleChange object as the value.

The Interface of SimpleChange is

```
export interface SimpleChange {  
  previousValue : any;  
  currentValue : any;  
  firstValue : boolean;  
}
```

✨ Use Case: To perform light operations depending on data-bound / Input properties

Pankaj ikhar | Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

✦ After Content Init Method

⇒ `public ngAfterContentChecked(): void;`

Use-case → It's called once after `ngDoCheck()`;

⇒ If you use one of `@ContentChild` to get `ContentChild`, they are initialized here.

☆ ✦ After Content Checked Method

⇒ `public ngAfterContentChecked(): void;`

Use-Case → Same as `ngDoCheck()` but for `ContentChild` properties

☆ ✦ After View Init Method

⇒ `public ngAfterViewInit(): void;`

Use-Case → It's called only once after first `ngAfterViewInit`

☆ ✦ After View Checked Method

⇒ `public ngAfterViewChecked(): void;`

→ Same as `ngDoCheck()` but for `ViewChildren`

✦ OnDestroy Method

→ Called immediately before Angular destroys the component/directives.

Let's Connect!

For interview preparation, short & long term mentorship, and design & architectural consulting.



Job interview preparation



Short & long term mentorship



Design & architectural consulting

Feel free to reach out:



greenleaf.educations@gmail.com



[/in/pankaj-ikkar](https://www.linkedin.com/in/pankaj-ikkar)



[@Pankajikkar](https://twitter.com/Pankajikkar)



Pankaj Ikkar 
Solution Architect & Mentor



Repost



Share



Save



Comment



Pankaj Ikkar

Solution Architect & Mentor

Mentorship • Interview Prep • Architecture Consulting

